Moving the Backend to Hugging Face Spaces

Why the backend could not stay where it was, what we considered, how we moved it, and what comes next.

Date: 29 September 2026 · Status: backend live; web app still needs to be pointed at it (section 8). Updated 30 September 2026 (section 10).

1. Summary

The Understand Kamba web app deploys to Vercel, but its Python backend (translation, speech recognition and voice) cannot run there: it needs about 5.5 GB of dependencies, about 7 GB of models and about 4.5 GB of memory, while Vercel functions are limited to 500 MB. Until the backend had a public home, the deployed site could not translate anything.

We compared paid hosts, free tiers and running it from a home PC, and moved the backend to a free Hugging Face Space on ZeroGPU hardware. The existing backend code runs there unchanged, wrapped in a small entry point. Getting it running took three fixes, each caused by how ZeroGPU and Gradio start apps. It is now live at https://lawmaluki-understand-kamba-api.hf.space and answers every endpoint in 1 to 2.5 seconds.

The last step is to set NEXT_PUBLIC_API_BASE_URL in Vercel and redeploy.

2. Where we started

The project has two parts:

- **Web app** (web/): a Next.js site. It runs in the visitor's browser and calls the backend over HTTP.
- **Backend** (app/): a FastAPI server running three models on the CPU: Meta NLLB-200 for translation, a w2v-BERT Kamba model for speech-to-text, and Meta's MMS Swahili voice for read-aloud.

Until now both ran only on a development PC, with the web app calling http://localhost:8000.

3. Why we had to move

3.1 Vercel could not build the project

The first Vercel deployment failed:

```
Error: Total bundle size (5513.37 MB) exceeds the maximum function size (500 MB).
```

Vercel saw requirements.txt at the top of the repository and tried to package the Python backend as a serverless function. PyTorch and transformers alone are about ten times Vercel's limit. Setting Vercel's Root Directory to web fixed the build: Vercel now deploys only the website, which is small.

3.2 The deployed site had nothing to talk to

With the website deployed, translation still failed with "Couldn't reach the translation server at http://localhost:8000". The browser console showed why:

```
Access to fetch at 'http://localhost:8000/translate' from origin
'https://understand-kamba.vercel.app' has been blocked by CORS policy:
Permission was denied for this request to access the `loopback` address space.
```

The deployed site had no backend address configured, so it fell back to localhost. Browsers block public websites from calling localhost, and even without that block, localhost means each visitor's own computer. The backend needed a public home.

3.3 What that home must provide

We measured the backend with all three models loaded:

Loaded	Memory
Python, PyTorch and transformers	0.3 GB
+ translation (NLLB)	2.5 GB
+ voice	2.7 GB
+ speech-to-text	4.5 GB

So the host needs:

- at least 4.5 GB of memory, ideally 8 GB for headroom;
- about 10 GB of disk for dependencies and models;
- a long-running server process (serverless platforms reload models on every cold start);
- an HTTPS address, because an HTTPS page cannot call an HTTP API.

4. Options we considered

Option	Cost	Verdict
Vercel	—	Impossible: 500 MB function limit.
Render Free, Starter or Standard	Free to about $25/month	Too little memory (512 MB to 2 GB).
Render Pro, 4 GB	About $85/month	Just under the 4.5 GB needed. Could work if the models were quantized to use less memory.
Render Pro Plus, 8 GB	About $175/month	Works, but costly for an experiment.
Railway, Koyeb or Fly.io free tiers	Free	Too little memory.
Google Colab	Free	Sessions end after a few hours.
Oracle Cloud Always Free	Free	Enough memory, but needs a credit card and manual server setup.
Development PC plus a Cloudflare Tunnel	Free	Fastest to set up, but only works while the PC is on, and the address changes each restart.

Option	Cost	Verdict
Hugging Face Space, Docker	Paid	We planned this, but the Docker option was marked Paid for this account.
Hugging Face Space, CPU Basic	Paid	Switching to it returned "402 Payment Required".
Hugging Face Space, ZeroGPU (Gradio)	**Free**	**Chosen.**

Render prices are as we understood them at the time; check render.com/pricing before relying on them.

Two of our assumptions turned out to be out of date: Hugging Face now charges for Docker Spaces and for CPU Basic hardware on this account. ZeroGPU with the Gradio SDK was the only free option left, and it came with its own rules (section 6).

5. The decision

We chose a free Hugging Face Space on ZeroGPU hardware because:

- **It costs nothing**, which suits a project still in its learning phase.
- **It has enough memory.** All three models load and run comfortably.
- **It is fast.** Models download from Hugging Face's own network at hundreds of MB per second, so a restart takes about a minute rather than many.
- **It gives a stable HTTPS address** that does not depend on anyone's PC being on.
- **The backend code did not need to change.** Everything Space-specific lives in a separate space/ folder.

We accepted these trade-offs:

- The Space sleeps after a couple of days without visitors; the next visit is slow while it wakes.
- Storage is temporary, so stats and ratings reset when the Space restarts.
- ZeroGPU has rules that ordinary servers do not (section 6), and they could change.

One deliberate design choice: **the models run on the CPU, not the GPU.** ZeroGPU lends a GPU for a few seconds at a time, but charges each visitor's daily quota: 2 minutes a day for anonymous visitors and 5 minutes for logged-in free accounts. Our web app calls the API anonymously, so heavy use would soon hit "quota exceeded" errors. On the CPU there is no quota, and the speed turned out to be good.

6. How we did it

6.1 Split the deployment

- **Website on Vercel,** with Root Directory set to web.
- **Backend on the Space.**

6.2 Create the Space

Created at huggingface.co/spaces/lawmaluki/understand-kamba-api with the Gradio SDK, ZeroGPU hardware, public visibility and a CC-BY-NC 4.0 licence (matching the non-commercial licences of the NLLB and MMS models). Public is required: the website calls the API from visitors' browsers, and a private Space would reject those calls.

6.3 Add a Space entry point, leaving the backend unchanged

The repository gained a space/ folder:

File	Purpose
space/server.py	Starts Gradio, adds a small test page at "/", and attaches the backend's existing API routes (/translate, /transcribe, /synthesize, /stats, /feedback, /health, /docs) to Gradio's server. It also loads the models in the background at startup, so the first visitor does not wait for downloads.
space/README.md	The Space's configuration header (SDK, Gradio version 6.28.0, Python 3.12, entry file, licence) and a short description.
space/requirements.txt	Backend dependencies, with PyTorch pinned to 2.13.0, the newest version ZeroGPU supports.

A deploy script, scripts/deploy_space.py, copies space/ and app/ into a temporary folder and uploads only that. The web app, docs, .env and local data never reach Hugging Face.

6.4 Three problems and their fixes

Before each upload we ran the entry point locally with the same Gradio version. The Space still needed three attempts, because ZeroGPU and Gradio behave differently on Hugging Face than on a PC.

Attempt 1: the Space crashed at startup. The logs showed two errors:

```
No @spaces.GPU function detected during startup
[Errno 98] error while attempting to bind on address ('0.0.0.0', 7860): address already in use
```

Reading the source of Hugging Face's spaces package showed that ZeroGPU hooks into Gradio's demo.launch(). At that moment it reports the app's GPU functions to the platform, and the platform refuses to start an app that declares none. Our first entry point started its own web server and never called demo.launch().

Fix: import spaces before anything that loads PyTorch, declare one small function marked @spaces.GPU that is never called, and start the app through demo.launch().

Attempt 2: the Space ran, but the API returned "405 Method Not Allowed". /health and /stats worked, but every POST, such as /translate, failed, and GET /translate returned a web page instead of our API.

On Spaces, recent Gradio versions run a Node.js server-side rendering (SSR) server on port 7860 in front of Python. It forwards only Gradio's own paths, so our routes never reached Python. This SSR server was also what held port 7860 in attempt 1.

Fix: launch with ssr_mode=False, so Gradio's Python server serves port 7860 directly.

Attempt 3: everything worked.

Cross-origin access needed no changes. Gradio has its own cross-origin (CORS) handling, which allows requests from any website when it is served from a public address such as *.hf.space. We confirmed this in

Gradio's source before relying on it, then tested it live.

6.5 Verification

We called the live API exactly as the website does, with the Vercel site as the origin:

Check	Result	Time
Cross-origin preflight from understand-kamba.vercel.app	Allowed	—
GET /health, GET /stats	200	0.3 s
English to Kikamba	"Nĩngũkwĩsũva ũete kĩw'ũ kuma ũsĩnĩ ũsu ũtanamba kũvika wĩoo."	1.6 s
Kikamba to English, round trip	"Please bring water from the river before evening."	1.1 s
Read aloud	WAV audio returned	2.3 s
Speech-to-text of a browser-style .webm recording	"ningukuisuvo uete kiu kuma usini usukuta na mbakuvika wioo"	1.4 s

In these tests the Space's CPU was faster than the development PC, and no GPU quota was used.

7. How it fits together now

Piece	Where it runs	Address
Website	Vercel	https://understand-kamba.vercel.app
API and models	Hugging Face Space (ZeroGPU hardware, CPU inference)	https://lawmaluki-understand-kamba-api.hf.space
API docs and test page	Same Space	/docs and /
Source code	GitHub	github.com/lawmaluki/understand-kamba

A visitor's browser loads the website from Vercel, then calls the Space directly for every translation, transcription, voice clip, rating and stats lookup. Vercel never talks to the Space itself.

To update the backend after changing app/ or space/:

```
python scripts/deploy_space.py
```

The Space rebuilds and restarts in about two minutes. Build and run logs are at huggingface.co/spaces/lawmaluki/understand-kamba-api under Logs.

8. Immediately outstanding

1. **Point the website at the Space.** In Vercel, go to Settings, then Environment Variables, and add NEXT_PUBLIC_API_BASE_URL with the value https://lawmaluki-understand-kamba-api.hf.space (no trailing slash). Then redeploy: the address is built into the site at build time.
2. **Replace the Hugging Face token.** The write token was pasted into a chat during setup, and it is also stored as the Space's HF_TOKEN secret (used to push voice recordings). Delete it at huggingface.co/settings/tokens and create a new one; ideally a fine-grained token with write access to only the Space and the kamba-voice-recordings dataset. Then log in again with hf auth login and update the HF_TOKEN secret in the Space's settings. Until the secret is updated, recordings cannot be saved.
3. **Merge the Space files.** space/, scripts/deploy_space.py and the voice-recording feature are on the feature/hf-space-backend branch, not yet on main.

9. Risks and limitations

Risk	Impact	Mitigation
The Space sleeps after a couple of days without traffic	The first visit afterwards waits while it wakes and loads models	Show a "waking up, please wait" message in the web app instead of an error (section 11).
Temporary storage	Stats and ratings reset on every restart	Move them to persistent storage (section 11).
Open API	Anyone who finds the address can use it. There is no login or rate limit, and Gradio's CORS handling accepts every website, so the API cannot be limited to the Vercel site	Add rate limiting; watch usage.
Relies on ZeroGPU and Gradio internals	The unused GPU function, ssr_mode=False and attaching routes to Gradio's server work today, but a Gradio or ZeroGPU update could break them	Gradio (6.28.0) and PyTorch (2.13.0) are pinned. Test locally before changing versions, and check the Space logs after each deploy.
Hugging Face policy changes	Docker and CPU Basic became paid during this work; ZeroGPU could change too. Free ZeroGPU hosting requires a verified email and an account older than 30 days, and allows at most 2 ZeroGPU Spaces.	The repository's Dockerfile keeps a paid-host option ready (see section 11).
Voice recordings are personal data	A person's voice can identify them; the project is responsible for storing the recordings safely and honouring deletion requests	Recordings are anonymous, consented, versioned and kept in a private dataset. Before collecting at scale, check obligations under Kenya's Data Protection Act 2019 (this document is not legal advice), and consider a contact email for deletion requests instead of public GitHub issues.
Non-commercial licences	NLLB-200 and the MMS voice are CC-BY-NC 4.0	Fine for learning and research; replace these models before any commercial use.

10. Update, 30 September 2026: bigger translation model and voice recordings

10.1 Translation now runs NLLB-3.3B on the GPU

We measured translation quality on 100 sentences of FLORES-200 (Meta's professionally translated test set, which includes Kamba), scoring with chrF++ (higher is better):

Kikamba to English	chrF++	BLEU	Time per sentence
NLLB-600M (original)	34.0	13.6	about 2.4 s
NLLB-200-distilled-1.3B on CPU	37.5	16.0	about 2.5 s
NLLB-3.3B on the ZeroGPU GPU (live)	**38.1**	**16.4**	**about 0.9 s**

English to Kikamba barely changed between models (chrF++ 29.7 to 30.5). Two community fine-tunes of the 600M model (KnoxDevelopers LoRA adapters) scored slightly worse than the original, so they were not used.

How it is set up:

- The Space setting NLLB_GPU_MODEL_ID=facebook/nllb-200-3.3B makes space/server.py load that model onto the GPU at startup, in half precision (about 7 GB).
- Calls from the website carry no visitor token, so ZeroGPU charges them all to one daily GPU quota shared by the whole Space. We confirmed this in the spaces package source: a token-less request that runs out of quota fails with "Space app has reached its GPU limit".
- So translation falls back automatically to NLLB-200-distilled-1.3B on the CPU (the NLLB_MODEL_ID setting) whenever the GPU call fails. The fallback model loads on first use, to save memory. 100 translations in a row all ran on the GPU, so the quota comfortably covers testing; heavy traffic will reach the fallback.
- /translate responses now include a model field naming the model that answered, so fallbacks are visible.

Quality is still well short of reliable. The larger model mainly buys speed and some better word choices; a Kikamba speaker's review remains the real test.

10.2 Collecting recordings for a real Kamba voice

No Kamba text-to-speech model exists. We compared Meta's voices for Kamba's relatives by having each read the same Kikamba sentences and checking how much the Kamba speech-to-text model misheard: Swahili 8%, Kikuyu 17%, Tharaka 22%. The Swahili voice stays for now. An authentic Kamba voice has to be trained on Kamba recordings, so the website now collects them:

- **A /contribute page** (linked from the home page and the player). Speakers agree to a plain-language consent statement, confirm they are 18+ and fluent, give their dialect (gender and age optional), then read sentences aloud, listen back, and submit, re-record or skip.
- **The sentences:** 1,044 Kamba sentences from FLORES-200 (CC-BY-SA 4.0), filtered to 40 to 140 characters with no digits or quotes (app/data/kikamba_prompts.tsv, built by scripts/build_prompts.py).
- **Storage:** recordings are converted to 16 kHz mono WAV and pushed every 5 minutes to the private dataset huggingface.co/datasets/lawmaluki/kamba-voice-recordings, in Hugging Face's audiofolder layout (audio files plus metadata.jsonl with the sentence, anonymous speaker ID, dialect, optional gender and age range, duration and consent version). Each Space start writes to its own folder, so a restart never overwrites earlier uploads.
- **Safeguards:** consent is required and versioned, contributors are anonymous (a random ID, no names or contact details), recordings must be 1 to 20 seconds and not silent, and each connection is limited to 120 uploads an hour. Contributors can ask for deletion by opening a GitHub issue with their ID.

A voice model typically needs one to three hours of clean speech from a speaker, so this will take many contributors or a few dedicated ones.

10.3 Update, 9 October 2026: OmniVoice becomes the main voice

OmniVoice (k2-fsa, April 2026) is a 600-language text-to-speech model whose training data included 14.7 hours of Kamba, most likely from Google FLEURS. Cloning a real Kamba speaker from FLEURS, it beat every voice tested before (characters misheard by the Kamba speech model, everyday sentences): OmniVoice cloned woman 2.0%, cloned man 3.2%, Swahili 8.9%, Kikuyu 18.2%. OmniVoice's own default voice failed on Kamba, so only cloning is used.

How it runs:

- The Space setting TTS_GPU_MODEL_ID=k2-fsa/OmniVoice makes space/server.py load it at startup, build clone prompts for the woman and man clips in app/data/voices on the CPU, and move the model to the GPU in full precision (about 2.4 GB). Casting it to float16 made it output silence.
- Each /synthesize request generates all its sentences in one GPU call, reserving GPU time in proportion to the text length (10 to 60 seconds) so short texts don't reserve a long slot from the shared quota.
- When the GPU can't be used, the Swahili MMS voice answers instead. The X-Voice-Model response header names the model that spoke, and the website's player shows "AI Kamba voice" or explains the Swahili fallback. Visitors choose Woman or Man.
- Live, each request takes about 7 to 9 seconds including ZeroGPU queueing. Translation (NLLB-3.3B) and the voice now draw on the same daily GPU quota.

Caveats: the FLEURS speakers recorded for research rather than to be a product voice, so the clips should be replaced with consenting speakers before wider use (app/data/voices/README.md); OmniVoice's weights are non-commercial (CC-BY-NC); and its accuracy figures may be flattered because the speech model used to score it may also have trained on FLEURS.

10.4 Update, 14 October 2026: a fine-tuned translator, corrections and lasting stats

- **Translation.** The full FLORES evaluation (docs/DOCUMENTATION.md, section 8) showed NLLB-3.3B was significantly *worse* than the distilled 1.3B model for English → Kikamba. The 1.3B model was then fine-tuned on existing public data on a free Kaggle GPU (training/), gaining about 10 chrF++ points for Kikamba → English. The Space now runs **lawmaluki/nllb-200-1.3B-kamba-ft** for both NLLB_GPU_MODEL_ID and NLLB_MODEL_ID. The model is public, so the Space needs no token to load it.
- **Corrections.** After a thumbs-down the app offers the model's other likely translations and a box to type the right one. Corrections go to the private dataset lawmaluki/kamba-translation-corrections (created automatically), are reviewed with training/review_corrections.py, and feed the next training run.
- **Stats.** Counts and ratings used to reset whenever the Space restarted. They are now synced to the private dataset lawmaluki/kamba-app-stats every 5 minutes and restored on start.
- **Token.** The old HF_TOKEN had been deleted after it was exposed, which broke the dataset uploads (voice recordings included) until a new write token was set as the Space secret.

11. What next

Short term

- Finish the three items in section 8.
- **Handle cold starts in the web app.** When a request times out or the Space is waking, show "The translation service is waking up; this can take a minute" and retry, instead of the current error.
- **Keep stats and ratings across restarts.** Options: a Hugging Face storage bucket attached to the Space (the create form offers one; check its current pricing), or a small free external database (for example a free Postgres tier) written to by app/stats.py.
- **Add basic rate limiting** per IP address to protect the free Space from abuse.

Medium term

- **Measure translation quality.** Build a test set of 30 to 50 sentences with Kikamba translations checked by a speaker, and score the current model against it. The ratings log (data/feedback.jsonl) is a good source of sentences that went wrong. Once ratings persist, review them regularly.
- **Grow the voice dataset, then train a Kamba voice.** Share the /contribute page with Kikamba speakers. Once there are one to three hours of clean speech from consistent speakers, fine-tune Meta's MMS voice (VITS) on it with a rented GPU, and compare it with the Swahili voice.
- **Watch GPU fallbacks.** The model field in /translate responses and the Space logs show how often translation falls back to the CPU model. If it happens often, consider Hugging Face PRO (more ZeroGPU quota) or paid hardware.
- **Automate deploys.** A GitHub Action could run scripts/deploy_space.py whenever app/ or space/ changes on main, so the backend never falls out of step with the code.

When usage grows

- **Move to paid, always-on hosting.** The repository's Dockerfile runs the backend on Render, Railway, Fly.io or a VPS with about 6 to 8 GB of memory, on CPU only (the GPU models are wired up only for the Space). Quantizing the translation model could let it fit a cheaper 4 GB plan.
- **Give visitors their own GPU quota** by letting them sign in with Hugging Face, so heavy use by one visitor doesn't use up the shared pool.
- **Replace the non-commercial models** before any commercial use.